

Log Aggregation, Analysis and Diagnostics Platform

Team 1: Emmanuel Dairo, Ahmed Ikram, Sarah Kelly, Sam Watts, Martin Kelly, Addie Tweed, David Hyacinth-Ngwu

Problem

Need to...

- Ingest different log sources across a network for ATMs into a homogenized form and aggregate them
- Assess logs for defined anomalies
- Analyse anomalies for patterns and correlations across different sources
- Visualise this information

Approach

Defined personas, user stories and requirements before allocating any development work.

This helped us have a shared understanding of how we wanted the system to work.

We then followed an agile approach, holding daily standups to keep track of progress.

For our tech stack, we went with Python for the backend, interacting with our React frontend via FastAPI. We also used SQLite for our database.

Team roles

Emmanuel - Leader / analysis

Ahmed - Log Ingestion / backend

Martin - Anomaly Detection / backend

Addie - Analysis / backend

Sarah - Menus / frontend

David - Details Page / frontend

Sam - Frontend / scrum master

Ingestion

Entrypoint for the rest of the system. Pulls logs and metrics from different sources, normalises them into a consistent format and stores them inside an SQLite database for the anomaly detection frontend.

- Timestamps converted to UTC
- Log levels standardised
- Fields mapped into unified tables:
 - **Events** for discrete log entries
 - **Metrics** for time-series numeric data.
- Anything that can't be parsed cleanly gets routed to the table **ingestion_errors**, as opposed to causing a crash

- Prometheus and GCP in CSV
- Kafka and app logs in JSON streams
- OS and hardware sensor metrics.

Each **source** has a dedicated parser, meaning new sources only require adding a new class.

All parsed data flows through a **normalisation** step.

Writes go out in batches through a 'writehelper' that handles SQLite lock contention with retry/backoff. Database runs in WAL mode for better concurrency.

PostgreSQL was considered, but given project scope; dealing with a few ATMs as opposed to hundreds of thousands, SQLite kept the stack simple, testable and self-contained.

Anomaly Detection

Follows on from ingestion by reading in from the database and identifying when something is going wrong across the network.

Each detector follows a checklist pattern, gathering signals from multiple sources and firing when all conditions from the spec are met.

Detected anomalies are written to the anomalies table with severity, explanation, correlation ID and JSON evidence ready to be consumed by the analysis layer and frontend.

A1: Network timeout cascade

A2: Cash cassette depletion

A3: JVM memory leak

A4: Container restart loop

A5: High response time spike —
threshold-based Kafka + ATM app
correlation

A6: OS memory pressure

A7: Malformed/out-of-order Kafka
events

Analysis

Responsible for giving more detail on anomalous activity within the system. Assigns an explanation for cause, the potential operational impacts and the recommended fixes for each anomaly.

Utilises fields from logs related to the anomaly; related to correlation IDs, such as from related client or ATM. Description varies and in large part corresponds to the type of anomaly it is (cash depletion, network timeout etc).

A1 Fields Example:

```
# '{"network_disconnect": true, "error_code_correct": true,
"timeout": true, "kafka_offline": true, "kafka_host_unavailable":
true, "terminal_timeout": true, "last_ts": "2026-03-
30T07:59:00+00:00"}
```

Orders analysis based on **ranking** of how critical they are.

Each analysis has its own function, so accounting for analysis of additional future anomaly types would only require adding a new function.

Numerous factors go into determining ranking:

- Anomaly type (Cash depletion, network timeout etc) are weighted differently in terms of how detrimental they are at a surface level. `'A1': 7, 'A4': 6, 'A3': 5, 'A2': 4, 'A6': 3, 'A5': 2, 'A7': 1`
- Time; older anomalies given higher priority
- Severity - `"CRITICAL": 3, "HIGH": 2, "MAJOR": 1, "WARNING": 0`
- Whether a transaction is in progress

Further Notable Backend

Data retention duration can be configured. REST endpoints let admins both configure how long data is kept (up to 365 days) and perform manual cleanup. APScheduler job that runs every hour.

Authorisation with JWT-based login with admin and user roles. all endpoints are properly guarded.

39 passing tests covering end-to-end: parsers, DB schema, concurrent writes, auth, analysis and all the main API endpoints

Display / Frontend

Due to our team's limited experience with frontend technologies, we decided to keep the UI simple, showing only the most important information.

We decided on a list containing high-level information about anomalies detected.

Dashboard

Starred

Completed

Admin Settings

Log Out

Anomalies Detected (7)

Detected anomalies across ATM and server systems, prioritised by severity.

Search by title...

- ⚠️ ATM offline due to network failure.**
ATM-GB-0003
CRITICAL Received: 2 hrs ago [View](#) ☆
- ⚠️ Container restart loop causing instability.**
SERVER
MAJOR Received: 24 hrs ago [View](#) ☆
- ⚠️ ATM out of service - cash cassettes exhausted.**
ATM-GB-0003
CRITICAL Received: 2 hrs ago [View](#) ☆
- ⚠️ JVM memory leak suspected - heap usage increasing.**
SERVER
MAJOR Received: 24 hrs ago [View](#) ☆

We also provided a page for detailed information about anomalies, showing their root cause, operational impact and recommended action.

ATM offline due to network failure.

Review analysis, understand the ATM issue, and track the recommended next steps.

☆ Mark as Starred ⌚ Mark as Completed

Root Cause

ATM-GB-0003 experienced a network timeout cascade. ATM application logs show a NETWORK_DISCONNECT event with ERR-0040, followed by a TIMEOUT at 30000ms. Kafka telemetry marked the ATM Offline and reported HOST_UNAVAILABLE. Terminal handler logs also recorded NETWORK_TIMEOUT, confirming a cross-source network failure.

Operation Impact

ATM unavailable for customer transactions due to network connectivity failure.

Recommended Action

Investigate the network link between the ATM and terminal handler, validate backend is reachable, and restore connections before returning the ATM to service.

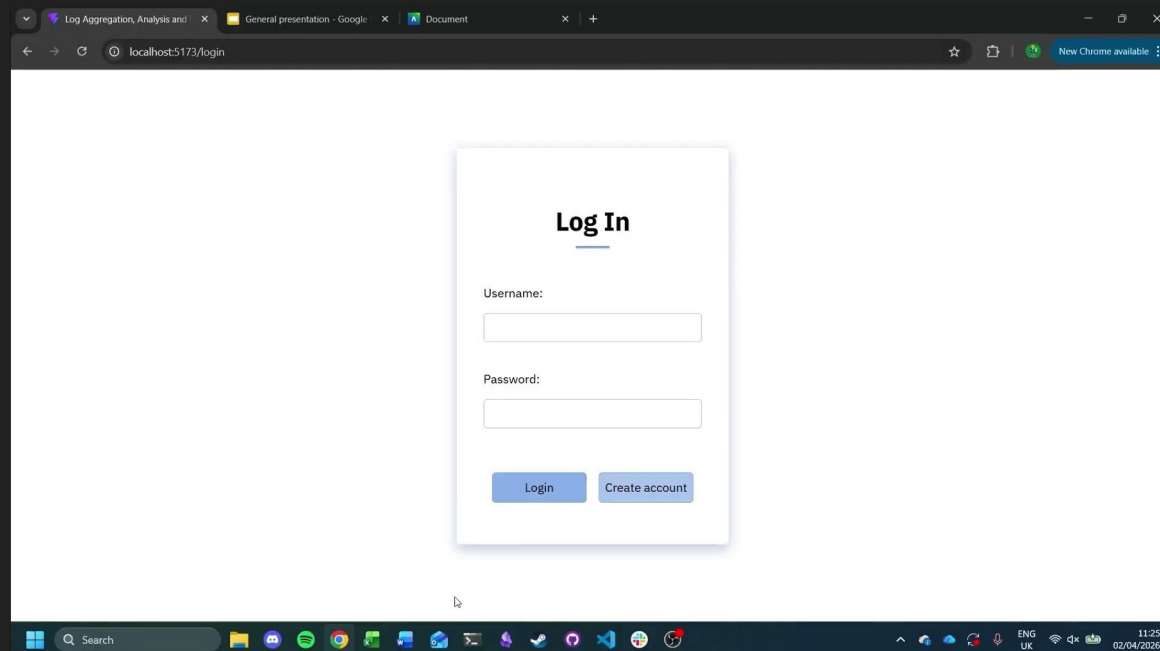
Details

ATM / Server: ATM-GB-0003

Severity: **CRITICAL**

Time Received: 2 Apr 2026, 07:49

DEMO



The screenshot shows a web browser window with the address bar displaying "localhost:5173/login". The browser has three tabs open: "Log Aggregation, Analysis and", "General presentation - Google", and "Document". The login form is centered on the page and contains the following elements:

- Log In**: A title with a blue underline.
- Username:** A label above a text input field.
- Password:** A label above a text input field.
- Login**: A blue button.
- Create account**: A blue button.

The Windows taskbar is visible at the bottom, showing the Start button, a search bar, and various application icons. The system tray on the right indicates the language is "ENG UK", the date is "02/04/2026", and the time is "11:25".

Problems brought up by Evaluation

3 Undergraduates: 2 Computer Science and 1 Business

- One participant acknowledged a lack of password policy. Most acknowledged the system would benefit from further security features like Multi-Factor Authentication or one time passwords.
- One user noted that the logo gave a false impression that it could be used as a home button. They also noted the lack of a back button.
- None of the users noticed the anomaly filtering functionality.
- One user noted that timestamps of anomalies should be involved in analysis ranking.

Post-Evaluation

Overall feedback was good. Usability and navigation beyond filtering and back buttons was well received. No confusion arose at anomaly or analysis information or the way it was presented.

One interesting note was one participant commenting on how they didn't understand the information within the analysis, but understood that relevant engineers likely would, which falls well into our Manager user story.

Implemented feedback:

- Got rid of logo
- Added back buttons
- Addition of password policy (minimum 6 characters)
- Moved anomaly filtering to be more noticeable
- Added time-based ranking to analysis
- Can remove an anomaly from "completed" status

Next Steps

Looking forward, there are a few features we would add:

- One of these is making starring user-based, instead of global.
- Another feature we would add would be to have predictive detection, using AI and pattern recognition to detect when anomalies could appear in the future.
- A further feature would be to allow real-time log ingestion, detecting anomalies as soon as they occur.
- We could also make the software less specific for ATMs, allowing any logging to be processed by the system.

Summary

We believe we've met all the requirements, implementing log ingestion, anomaly detection, analysis and a visual interface to display this information.

Furthermore, we implemented some additional features such as custom data retention length, anomaly display filtering, and admin and user permissions, and data access.

Thanks!